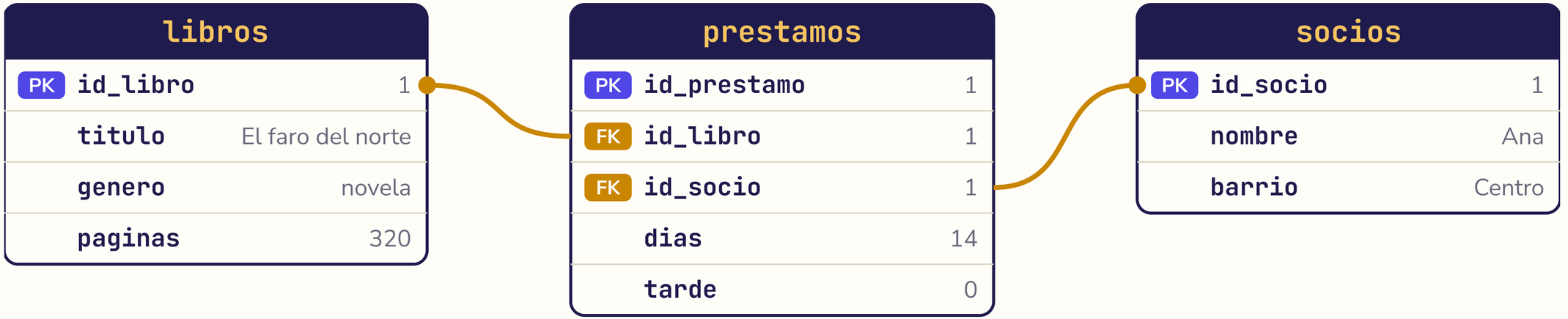
IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Bases de datos relacionales y SQL

Tablas conectadas y el idioma para hacerles preguntas

Sprint 3 · S03_01 · presentación (40 min)

Tablas conectadas entre sí



Tabla, fila y columna

Una **tabla** es como una hoja de cálculo. Cada **fila** es un registro (un préstamo); cada **columna**, un dato (los días).

Clave primaria (PK)

Identifica **cada fila de forma única**, como un DNI. En **libros** es **id_libro**: no hay dos iguales.

Clave foránea (FK)

Una columna que guarda el número de **otra tabla**. **id_libro** de **prestamos** apunta a la fila de **libros**.

LA IDEA · POR QUÉ SE SEPARA EN TABLAS

Un dato, un solo sitio

Todo en una sola tabla (extracto)

id_prestamo	titulo	socio	dias
1	El faro del norte	Ana	14
2	El Faro del Norte	Beto	24
3	Atlas de volcanes	Carla	10
8	el faro del norte	Carla	30

Con tablas separadas (extracto)

id_prestamo	id_libro	socio	dias
1	1	Ana	14
2	1	Beto	24
3	2	Carla	10
8	1	Carla	30

Contar préstamos por título, con todo en una tabla

El Faro del Norte: **1**

El faro del norte: **1**

el faro del norte: **1**

Un solo libro sale como **tres** libros.

Contar préstamos por libro, con tablas separadas

id_libro = 1 (El faro del norte): **3** préstamos.

El título se escribe **una vez**, en **libros**: si cambia, se corrige **una vez**.

LA IDEA · CONSULTAS BÁSICAS

SELECT: preguntar a una tabla

```
3 SELECT titulo, paginas
1 FROM libros
2 WHERE paginas > 150
4 ORDER BY paginas DESC
5 LIMIT 3;
```

titulo	paginas
Sombras en el puerto	410
El faro del norte	320
Plantas de la isla	220

1. **`FROM libros`**
   De qué tabla se parte: 7 filas.
2. **`WHERE paginas > 150`**
   Qué filas se quedan: 4 de 7.
3. **`SELECT titulo, paginas`**
   Qué columnas se ven: solo dos.
4. **`ORDER BY paginas DESC`**
   En qué orden: de más a menos páginas.
5. **`LIMIT 3`**
   Cuántas filas como máximo: 3.

LA IDEA · UNIR TABLAS

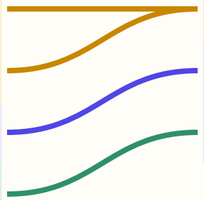
JOIN: pegar los datos de otra tabla

prestamos (extracto)

id_prestamo	dias	id_libro
1	14	1
2	24	1
3	10	2
4	28	5

libros (extracto)

id_libro	titulo
1	El faro del norte
2	Atlas de volcanes
5	Sombras en el puerto



→

Resultado del JOIN

id_prestamo	titulo	dias
1	El faro del norte	14
2	El faro del norte	24
3	Atlas de volcanes	10
4	Sombras en el puerto	28

```sql
SELECT p.id_prestamo, l.titulo, p.dias
FROM prestamos p
JOIN libros l ON l.id_libro = p.id_libro
ORDER BY p.id_prestamo
LIMIT 4;
```

Cómo se lee

prestamos p y **libros l** son **apodos** (alias) para escribir menos.

ON dice qué columnas **coinciden**: la clave foránea con su clave primaria.

LA IDEA · LO QUE NO TIENE PAREJA

LEFT JOIN: no perder ninguna fila de la izquierda

JOIN: 10 filas

Solo los libros **con préstamos**. «Guía de senderos» **desaparece**.

LEFT JOIN: 11 filas

Todos los libros. Si no hay préstamo, lo de la derecha sale **NULL**, que significa **vacío**.

Las dos últimas filas del LEFT JOIN

titulo	id_prestamo
Plantas de la isla	9
Guía de senderos	*NULL*

```
SELECT l.titulo, p.id_prestamo
FROM libros l
LEFT JOIN prestamos p ON p.id_libro = l.id_libro
WHERE p.id_prestamo IS NULL;
```

Libros que nadie se ha llevado: sale 1 fila, Guía de senderos. Se pregunta con IS NULL, nunca con = NULL.

LA IDEA · RESUMIR POR GRUPOS

GROUP BY: un resumen por grupo

id_socio = 1 · Ana

0 1 0

COUNT(*) = 3

pct_tarde = 33,3

id_socio = 2 · Beto

1 0

COUNT(*) = 2

pct_tarde = 50,0

id_socio = 3 · Carla

0 0 1

COUNT(*) = 3

pct_tarde = 33,3

id_socio = 4 · Dani

0 0

COUNT(*) = 2

pct_tarde = 0,0

```sql
SELECT id_socio, COUNT(*) AS prestamos,
       ROUND(100 * AVG(tarde), 1) AS pct_tarde
FROM prestamos
GROUP BY id_socio;
```

Funciones de resumen

COUNT cuenta, **SUM** suma, **AVG** hace la media, **MIN** y **MAX** dan el menor y el mayor.

Truco: **tarde** vale 0 o 1, así que su **media es la proporción**. Beto: (1 + 0) / 2 = 0,5, o sea 50 % tarde.

LA IDEA · FILTRAR ANTES Y DESPUÉS DE AGRUPAR

WHERE filtra filas; HAVING filtra grupos

```sql
SELECT id_socio, COUNT(*) AS prestamos
FROM prestamos
WHERE dias > 8
GROUP BY id_socio
HAVING COUNT(*) >= 2;
```

Dos filtros, dos momentos

WHERE actúa **antes** de agrupar y filtra **filas**.

HAVING actúa **después** y filtra **grupos**.

En un **WHERE**, **COUNT(*)** da error: aún no hay grupos.

`FROM prestamos` — 10 filas → `WHERE dias > 8` — 8 filas → `GROUP BY id_socio` — 4 grupos → `HAVING COUNT(*) >= 2` — 2 grupos → `SELECT …` — 2 filas: socios 1 y 3

El ordenador sigue este camino: la tabla se va **reduciendo** paso a paso, primero en filas y luego en grupos.

LA IDEA · CAMBIAR LOS DATOS

CRUD: crear, leer, cambiar y borrar

Create	`INSERT INTO`	añade una fila nueva
Read	`SELECT`	lee, sin cambiar nada
Update	`UPDATE`	cambia filas (con **WHERE**)
Delete	`DELETE FROM`	borra filas (con **WHERE**)

Sin WHERE se toca TODO

UPDATE prestamos_copia SET dias = 14; cambió **10 de 10** filas.

Con **WHERE id_prestamo = 3**, cambió **1**.

1 · Primero un SELECT

Prueba el **mismo WHERE** en un **SELECT**: ves qué filas tocarías.

2 · Siempre en una copia

Practica en **prestamos_copia**, **nunca** en la tabla original.

3 · Una red de seguridad

BEGIN … **ROLLBACK**: un **DELETE** sin **WHERE** dejó **0** filas y el **ROLLBACK** devolvió las **10**.

En DB Browser los cambios no quedan guardados hasta pulsar **Escribir cambios** (Ctrl+S); **Revertir cambios** los deshace.

PARA CONSULTAR · SCRIPT 04

Subconsultas y CTE: una consulta dentro de otra

Escalar: devuelve un solo valor.

```sql
SELECT titulo FROM libros
WHERE paginas > (SELECT AVG(paginas) FROM libros);
```

La media de páginas es **211,4**: salen **3** libros.

Con IN: devuelve una lista.

```sql
SELECT id_prestamo, dias FROM prestamos
WHERE id_libro IN (SELECT id_libro FROM libros
                   WHERE genero = 'ciencia');
```

Salen **3** préstamos, los de libros de ciencia.

CTE: ponerle nombre a un paso intermedio.

```sql
WITH largos AS (
    SELECT id_libro, COUNT(*) AS n_largos
    FROM prestamos WHERE dias > 14 GROUP BY id_libro)
SELECT l.titulo, g.n_largos
FROM largos g JOIN libros l ON l.id_libro = g.id_libro
ORDER BY l.titulo;
```

titulo	n_largos
El faro del norte	2
Sombras en el puerto	2

Con **WITH** la consulta se lee de arriba abajo: primero **largos**, luego el resultado.

PARA CONSULTAR · SCRIPT 06

Reglas e índices: el dato malo falla al entrar

```
RESTRICCIONES
CREATE TABLE multas (
    id_multa    INTEGER PRIMARY KEY,
    id_prestamo INTEGER NOT NULL
                REFERENCES prestamos(id_prestamo),
    euros       REAL NOT NULL CHECK (euros > 0)
);
```

Regla	Evita...
PRIMARY KEY	filas repetidas
REFERENCES	apuntar a algo que no existe
NOT NULL	campos vacíos
CHECK	valores absurdos

Índice: el «índice de un libro»

Sin índice, la base lee **todas** las filas: **SCAN prestamos**.

Con **CREATE INDEX idx_prestamos_socio ON prestamos(id_socio)**, va directa: **SEARCH prestamos USING INDEX idx_prestamos_socio**.

Administración mínima

Copias de seguridad, permisos y **no tocar producción** sin necesidad.

EL RETO · PRIMER MODELO EN PRODUCCIÓN

Esta presentación prepara el paso 1 del reto

Paso 1 · Entender el problema y los datos

Mirar la tabla de reservas con **SELECT**.

Calcular con **GROUP BY** el porcentaje de cancelaciones **por canal**: la media de una columna de 0 y 1.

Escribir en una frase qué se predice y para qué.

No entra en el reto, pero lo usaréis

CRUD, modelado e índices: en el Sprint 5 se hace CRUD desde Python con **sqlite3**.

NoSQL: se ve en esta misma jornada, en 15 minutos.

El día 1 del sprint: 200 minutos

10 min	40 min	105 min	15 min	30 min
Planning	**Esta presentación**	Scripts SQL 01–06 en DB Browser	Demo NoSQL	Cuaderno NoSQL

ANTES DE ABRIR DB BROWSER

Lo que hay que llevarse

Tablas y claves

Tablas **conectadas** por claves: cada dato se escribe **una vez**.

Preguntar con SQL

SELECT, **JOIN** y **GROUP BY** responden preguntas; **LEFT JOIN** conserva lo que no tiene pareja.

Cambiar con cuidado

Sin **WHERE**, **UPDATE** y **DELETE** tocan **toda** la tabla: copia, **SELECT** antes y **ROLLBACK**.

Ahora: scripts SQL 01 a 06 en parejas, 105 minutos, con Ctrl+Intro para ejecutar solo lo seleccionado.